Práctica 7 – Algoritmos Golosos (Greedy)

Los objetivos de esta práctica son:

- Aprender a identificar estrategias greedy.
- Demostrar la correctitud de estas estrategias.
- Comparar con otras estrategias: backtracking y programación dinámica.

Los ejercicios marcados con el símbolo ★ constituyen un subconjunto mínimo de ejercitación. Sin embargo, aconsejamos fuertemente hacer todos los ejercicios.

Los ejercicios marcados con el símbolo ℹ tienen una ayuda al final de la guía.

Ejercicio 1 *(Caramelos)* ★

Andrea tiene un conjunto de caramelos C de tamaño n. Cada caramelo tiene un valor de sabor c_i. La mamá no le deja comerse más de $k \leq n$ caramelos, así que Andrea decide comerse los k caramelos que maximicen la suma de sabores. Está pensando cómo encontrar el subconjunto S de C de tamaño k de suma máxima. En otras palabras, quiere encontrar $\max_{S \subseteq X, |S|=k} \sum_{s \in S} s$.

a) Primero se le ocurre hacer un algoritmo de backtracking: define las soluciones válidas como las secuencias de k caramelos distintos de C, y las parciales como los prefijos de soluciones válidas. Para construir una nueva solución parcial, elige un caramelo de entre todos los que no están en la solución parcial actual S, y lo agrega. En otras palabras, se define la siguiente función recursiva:

$$\mathsf{máxSuma}_{C,k}(S) := \begin{cases} \sum_{s \in S} s & \text{si } |S| = k, \\ \max_{c \in C \setminus S} \{\mathsf{máxSuma}_{C,k}(S \cup \{c\})\} & \text{sino.} \end{cases}$$

Demostrar que

$$\mathsf{máxSuma}_{C,k}(\emptyset) = \max_{S \subseteq X, |S|=k} \sum_{s \in S} s.$$

ℹ

b) Dar la complejidad temporal y espacial de la implementación con backtracking de la función $\mathsf{máxSuma}_{C,k}(S)$.

c) Andrea planeaba realizar este algoritmo para encontrar el mejor subconjunto y después comérselo, pero cada vez que construía una solución parcial nueva, su gula le ganaba: siempre se comía el caramelo que mejor sabor tenía entre los que le quedaban. Convencerse de que la siguiente función calcula cuánto sabor acumulado consumió Andrea:

$$\mathsf{máxSuma}^{\text{greedy}}_{C,k}(S) := \begin{cases} \sum_{s \in S} s & \text{si } |S| = k, \\ \mathsf{máxSuma}^{\text{greedy}}_{C,k}\left(S \cup \left\{ \max_{c \in C \setminus S} \{c\} \right\}\right) & \text{sino.} \end{cases}$$

d) Para $C := \{1, 2, 3\}$ y $k := 2$, dibujar el árbol de llamadas recursivas de $\mathsf{máxSuma}_{C,k}(\emptyset)$ y de $\mathsf{máxSuma}^{\text{greedy}}_{C,k}(\emptyset)$. ¿Es alguno subárbol del otro?

e) Demostrar que en realidad está bien lo que hizo Andrea. Es decir, demostrar que

$$\mathsf{máxSuma}^{\text{greedy}}_{C,k}(\emptyset) = \mathsf{máxSuma}_{C,k}(\emptyset) = \max_{S \subseteq X, |S|=k} \sum_{s \in S} s.$$



f) Implementar la función $\text{máxSuma}_{C,k}^{\text{greedy}}$ en el lenguaje de programación que más les guste. ¿Cuál es la complejidad temporal y espacial? ¿Cómo se compara con el algoritmo de backtracking del Item b)?

g) Extender el algoritmo del Item f) para que también devuelva uno de los subconjuntos S que maximiza la suma.

h) Dar una implementación del algoritmo del Item f) con complejidad temporal $O(n \log n)$. Codearla en su lenguaje favorito.

i) (Opcional) Dar una implementación del algoritmo del Item f) con complejidad temporal $O(n \log k)$.

Ejercicio 2 *(Ruta eficiente)* ★

Tomás quiere viajar de Buenos Aires a Mar del Plata en su flamante Renault 12. Como está preocupado por la autonomía de su vehículo, se tomó el tiempo de anotar las distintas estaciones de servicio que se encuentran en el camino. Modeló el mismo como un segmento de 0 a M, donde Buenos Aires está en el kilómetro 0, Mar del Plata en el M, y las distintas estaciones de servicio están ubicadas en los kilómetros $0 = x_1 \leq x_2 \leq \dots x_n \leq M$.

Razonablemente, Tomás quiere minimizar la cantidad de paradas para cargar nafta. Él sabe que su auto es capaz de hacer hasta C kilómetros con el tanque lleno, y que al comenzar el viaje este está vacío.

a) Proponer un algoritmo greedy que indique cuál es la cantidad mínima de paradas para cargar nafta que debe hacer Tomás, y que aparte devuelva el conjunto de estaciones en las que hay que detenerse. Probar su correctitud comparando la solución G devuelta por el algoritmo con otra solución diferente S, tomando el primer índice i de parada en el que difieren G y S, y demostrando que S puede ser modificada para que no difieran en ese índice sin aumentar la cantidad de paradas.

b) Dar una implementación de complejidad temporal $O(n)$ del algoritmo del Item a).

Ejercicio 3 *(Suma subconjuntos)* ★

Tenemos un conjunto de enteros S y un valor t. Queremos encontrar el subconjunto $S' \subseteq S$ más chico tal que $\sum_{s \in S'} s = t$.

A Gervasio le parece obvia la solución: simplemente ir agarrando el número de S más grande que no supere a t hasta llegar a t. Si no se puede llegar a t de esta forma, entonces no hay solución.

Mostrarle a Gervasio un conjunto S y un valor t para el cual esta estrategia no es correcta.

Ejercicio 4 *(Vertex cover)* ★

En el problema de *cubrimiento de aristas por vértices (vertex cover)* tenemos de entrada un grafo G, y se nos pide determinar la mínima cantidad de vértices que hay que sacar de G tal que el grafo resultante no tenga ninguna arista. A Ayelén se le ocurrió la siguiente estrategia greedy para el problema: sacar iterativamente el vértice de grado máximo entre los que quedan hasta que todos tengan grado 0.

a) Demostrar que esta estrategia no funciona para el caso general, mostrando un grafo en el que el cubrimiento mínimo sea más chico que el cubrimiento conseguido con este algoritmo.

b) Implementar esta estrategia en su lenguaje de programación favorito, y probarla con el grafo del inciso anterior.

Ejercicio 5 *(Deadlines)* ★

Se tiene un conjunto de n tareas $T \coloneqq \{t_1, t_2, \ldots, t_n\}$, donde cada tarea t_i tiene asociado un *deadline* $d_i \in \mathbb{N}$ que representa el tiempo límite antes del cual debe completarse la tarea. Debemos respetar las siguientes restricciones:

- Cada tarea requiere exactamente **una unidad de tiempo** para completarse.
- Solo se puede ejecutar **una tarea a la vez** (no existe paralelismo).
- El tiempo comienza en $t = 0$ y avanza en unidades discretas: $t \in \{0, 1, 2, \ldots\}$.
- Una tarea t_i se considera completada exitosamente si y solo si termina su ejecución en un tiempo $f_i \leq d_i$, donde f_i es el tiempo de finalización de la tarea.
- Una vez iniciada una tarea, debe completarse sin interrupciones (no hay *preemption*).
- Las tareas pueden ejecutarse en cualquier orden.

Por ejemplo, considere $n \coloneqq 4$ tareas con *deadlines* $D \coloneqq \{2, 1, 3, 2\}$. Una solución óptima sería: $O \coloneqq \{t_2, t_1, t_3\}$. Esta planificación completa 3 tareas exitosamente. La tarea t_4 no puede ejecutarse sin violar su *deadline*.

a) Proponer un algoritmo greedy que determine una planificación (orden de ejecución) que **maximice el número total de tareas** que pueden completarse respetando sus respectivos deadlines.
b) Demuestre la optimalidad.

Ejercicio 6 *(Producto escalar)* ★

Dados dos vectores $v = (v_1, v_2, \ldots, v_n)$ y $w = (w_1, w_2, \ldots, w_n)$ en $\mathbb{R}^n$, queremos encontrar permutaciones $\sigma, \tau : \{1, \ldots, n\} \to \{1, \ldots, n\}$ de sus coordenadas que minimicen el producto escalar entre los vectores reordenados. Es decir, queremos encontrar σ y τ que minimicen

$$\langle v_\sigma, w_\tau \rangle \coloneqq \sum_{i=1}^{n} v_{\sigma(i)} \cdot w_{\tau(i)},$$

donde $v_\sigma = (v_{\sigma(1)}, \ldots, v_{\sigma(n)})$ y $w_\tau = (w_{\tau(1)}, \ldots, w_{\tau(n)})$

a) Proponer un algoritmo greedy que encuentre las permutaciones óptimas.
b) Demostrar la optimalidad.

Ejercicio 7 *(Demoras)*

Se tiene una lista n de tareas, donde cada tarea $i = 1, \ldots, n$ toma $t_i > 0$ unidades de tiempo en ser completada. Estas tareas deben ser resueltas secuencialmente, es decir, no se pueden resolver dos tareas en paralelo. Dada una permutación de las tareas $t_\pi = (t_{\pi(1)}, \ldots, t_{\pi(n)})$, se define la demora de una tarea como el tiempo que se debe esperar para que la i-ésima tarea sea completada: $demora_\pi(i) = \sum_{j=1}^{i} t_{\pi(j)}$. Nos interesa elegir un orden en el cual resolver las tareas de manera que se minimice la sumatoria de demoras de todas: $\sum_{i=1}^{n} demora_\pi(i)$.

a) Proponer un algoritmo greedy que resuelva el problema.
b) Demostrar la optimalidad de la estrategia greedy.

Ejercicio 8 *(Max mex)*

Se define la función $\text{mex} : \mathcal{P}(\mathbb{N}) \to \mathbb{N}$ como

$$\text{mex}(X) = \min_{j \in \mathbb{N}, j \notin X} \{j\}.$$

Intuitivamente, mex (Mínimo EXcluido) devuelve, dado un conjunto X, el menor número natural que no está en X. Por ejemplo, $\text{mex}(\{0,1,2\}) = 3$, $\text{mex}(\{0,1,3\}) = 2$ y $\text{mex}(\{1,2,3,\ldots\}) = 0$.

Dado un vector de números $(a_1, \ldots, a_n)$ queremos encontrar la permutación $(b_1, \ldots, b_n)$ de los mismos que maximice

$$\sum_{i=1}^{n} \text{mex}(\{b_1, \ldots, b_i\}).$$

Por ejemplo, si el vector es $(3, 0, 1)$, podemos ver que la mejor permutación es $(0, 1, 3)$, que alcanza un valor de

$$\text{mex}(\{0\}) + \text{mex}(\{0,1\}) + \text{mex}(\{0,1,3\}) = 1 + 2 + 2 = 5.$$

a) Proponer un algoritmo greedy que resuelva el problema y demostrar su correctitud.
b) Dar una implementación del algoritmo del inciso anterior con complejidad temporal $O(n)$.

Ejercicio 9 *(Bicoloreo)*

Recordemos el problema de k-coloreo de un grafo G: determinar si existe una asignación de cada vértice a uno de k colores tal que todo par de vértices adyacentes tienen distinto color.

El algoritmo greedy más conocido define un orden arbitrario de los vértices $(v_1, \ldots, v_n)$ y pinta los vértices del primero al último, cada uno con el primer color todavía disponible. Ver Algoritmo 1.

Hay grafos en los que el Algoritmo 1 da un coloreo *óptimo* para cualquier orden inicial de los vértices, en el sentido que usa la mínima cantidad de colores.

a) Mostrar un grafo G para el cual exista un orden de los vértices tal que el Algoritmo 1, usando ese orden, no determine correctamente la mínima cantidad de colores que se necesitan para G.
b) Demostrar que en los grafos bipartitos completos, el Algoritmo 1 da un coloreo óptimo para cualquier orden de los vértices.
c) ¿Cuál es la mejor complejidad temporal que se les ocurre con la que se puede implementar el Algoritmo 1?

Algoritmo 1 Coloreo greedy

```
1 color ← (∞, ..., ∞)
2 for v_i ∈ V(G) do
3     color(v_i) ← mex_{w∈N(v_i)}{color(w)}
```

Ejercicio 10 *(Caballeros)*

El reino de Tedealand cuenta con $2n$ caballeros, que debe dividir entre las torres Norte (N) y Sur (S). Cada caballero tiene un costo N_i para transportarse a la torre N y S_i para transportarse a la torre S. Tu objetivo es dividir a los $2n$ caballeros entre las dos torres, de manera que cada una tenga **exactamente** n caballeros, minimizando el costo total. Es decir, siendo $TorreN$ el conjunto de soldados de la torre N y $TorreS$ el de soldados en la torre S, minimizar $costo(TorreN, TorreS) = \sum_{i \in TorreN} N_i + \sum_{i \in TorreS} S_i$.

a) Considerar el algoritmo greedy que manda a $TorreN$ a los caballeros de menor N_i. Mostrar un contra-ejemplo que demuestre que esta estrategia no es óptima.
b) Plantear un algoritmo greedy que divida a los caballeros de manera óptima.
c) Demuestre que ese algoritmo greedy es correcto. La demostración debe mostrar que no existe ninguna solución con menor costo que la obtenida por el algoritmo greedy.

Ejercicio 11 *(División pandémica)*

En medio de una pandemia, la Escuela de Aulas Grandes y Ventiladas quiere implementar un protocolo especial de distanciamiento social que tenga en cuenta que la escuela no tiene restricciones de espacio. El objetivo es separar a cada curso en dos subcursos a fin de reducir la cantidad de pares de estudiantes que sean muy cercanos, dado que se estima que estos estudiantes tienen dificultades para respetar tan buscado distanciamiento. Para este fin, en el protocolo se estableció que cada curso que tenga c parejas de estudiantes cercanos tiene que dividirse en dos subcursos, cada uno de los cuales puede tener a lo sumo $c/2$ parejas de estudiantes cercanos. Notar que no importa si un subcurso queda con más estudiantes que otro.

Formalmente, para cada curso contamos con un conjunto de estudiantes E y su conjunto C de pares de estudiantes cercanos. Luego, una partición (A, B) de E es una *solución factible para* (E, C) cuando $|(A \times A) \cap C| \leq |C|/2$ y $|(B \times B) \cap C| \leq |C/2|$. Por ejemplo, si $E \coloneqq \{1, 2, 3, 4\}$ y $C \coloneqq \{1\text{-}2, 2\text{-}3, 3\text{-}4\}$, entonces $(\{1, 3, 4\}, \{2\})$ y $(\{2, 4\}, \{1, 3\})$ son soluciones factibles.

a) Especificar el problema descrito definiendo cuál es la instancia (i.e. cuáles son los datos de entrada y qué condiciones satisfacen) y cuál es el resultado esperado (i.e., cuáles son los datos de salida y qué condiciones satisfacen).
b) Demostrar que para toda instancia existe un resultado esperado que satisface las condiciones definidas por el protocolo.
c) A partir de la demostración del inciso anterior, diseñar un algoritmo que encuentre una solución factible en tiempo lineal en función del tamaño de la entrada definido en el inciso a).

Ejercicio 12 *(Parejas de baile)*

Tenemos dos conjuntos de personas y para cada persona sabemos su habilidad de baile. Queremos armar la máxima cantidad de parejas de baile, sabiendo que para cada pareja debemos elegir exactamente una persona de cada conjunto de modo que la diferencia de habilidad sea menor o igual a 1 (en módulo). Además, cada persona puede pertenecer a lo sumo a una pareja de baile. Por ejemplo, si tenemos un multiconjunto con habilidades $\{1, 2, 4, 6\}$ y otro con $\{1, 5, 5, 7, 9\}$, la máxima cantidad de parejas es 3. Si los multiconjuntos de habilidades son $\{1, 1, 1, 1, 1\}$ y $\{1, 2, 3\}$, la máxima cantidad es 2.

a) Considerando que ambos multiconjuntos de habilidades estan ordenados en forma creciente, observar que la solución se puede obtener recorriendo los multiconjuntos en orden para realizar los emparejamientos.
b) Diseñar un algoritmo goloso basado en el Item a) que recorra una única vez cada multiconjunto. Explicitar la complejidad temporal y espacial auxiliar.
c) Demostrar que si se puede formar una pareja usando la persona con menor habilidad de baile en cada multiconjunto, entonces existe una solución óptima que contiene esa pareja.
d) Demostrar que el algoritmo dado en el Item b) es correcto usando el lema demostrado en el Item c).

Ejercicio 13 *(Suma golosa (difícil))*

Queremos encontrar la suma de los elementos de un multiconjunto de números naturales. Cada suma se realiza exactamente entre dos números x e y y tiene costo $x + y$.

Por ejemplo, si queremos encontrar la suma de $\{1, 2, 5\}$ tenemos 3 opciones:

- $1 + 2$ (con costo 3) y luego $3 + 5$ (con costo 8), resultando en un costo total de 11;
- $1 + 5$ (con costo 6) y luego $6 + 2$ (con costo 8), resultando en un costo total de 14;
- $2 + 5$ (con costo 7) y luego $7 + 1$ (con costo 8), resultando en un costo total de 15.

Queremos encontrar la forma de sumar que tenga costo mínimo, por lo que en nuestro ejemplo la mejor forma sería la primera.

a) Explicitar una estrategia golosa para resolver el problema.
b) Demostrar que la estrategia propuesta resuelve el problema.
c) Implementar esta estrategia en un algoritmo iterativo. **Nota:** el mejor algoritmo simple que conocemos tiene complejidad $\mathcal{O}(n \log n)$ y utiliza una estructura de datos que implementa una secuencia ordenada.

Ayudas

Ejercicio 12

Puede ayudar escribir el algoritmo como una función recursiva, y encontrar una semántica apropiada para esta función.

Ejercicio 11

Hacer inducción en la cantidad de estudiantes. Para el paso inductivo, considerar que si les estudiantes se asignan iterativamente a los subcursos, entonces conviene enviar a cada estudiante al subcurso que tenga la menor cantidad de estudiantes cercanos a elle.

Ejercicio 10

Considerar el valor de la resta $N_i - S_i$. ¿Qué representa?

Ejercicio 8

¿Cuál el máximo valor que puede tomar $\text{mex}(X)$ si X tiene n elementos? Si $X \subseteq Y$, ¿Qué pasa con los valores $\text{mex}(X)$ y $\text{mex}(Y)$?

Ejercicio 1

Demostrar por inducción en $j := k - |S|$ que $\mathsf{máxSuma}_{C,k}(S)$ es la máxima suma de un subconjunto de C de k elementos que contiene a S. En otras palabras, demostrar que

$$\mathsf{máxSuma}_{C,k}(S) = \max_{S' \subseteq C, |S'|=k, S' \supseteq S} \sum_{s \in S'} s.$$

Ejercicio 1

Demostrar por inducción en $j := k - |S|$ que $\mathsf{máxSuma}^{\text{greedy}}_{C,k}(S)$ es la máxima suma de un subconjunto de C de k elementos que contiene a S. En otras palabras, demostrar que

$$\mathsf{máxSuma}^{\text{greedy}}_{C,k}(S) = \max_{S' \subseteq C, |S'|=k, S' \supseteq S} \sum_{s \in S'} s.$$